TECHNICAL DEEP DIVE

CSim Renderer Architecture and Execution Report

A code-grounded examination of frame orchestration, token generation, resource ownership, OpenGL execution, Canvas presentation, UI batching, testability, performance behavior, and current architectural risks.

CORE FINDING

CSim uses a deliberately small, ordered token renderer. Production drawables emit data-oriented commands; the backend resolves opaque resource handles; GLDevice executes OpenGL synchronously. The design is well matched to one cellular-automata surface plus lightweight overlays, but queue overflow, state assumptions, and dormant scaffolding deserve targeted cleanup.

Repository	C:\Users\gravi\.grok\worktrees\projects-csim-copy\csim
Code baseline	Live source tree inspected on 2026-08-02
Validation	Existing Release headless suite executed: 0 failures
Runtime path	Windows + OpenGL 3.3 core profile + GLFW + GLEW
Report boundary	Read-only source analysis; no live OpenGL smoke test

Prepared as a durable engineering reference. Current implementation is authoritative where historical notes or superseded R8-only presentation documents disagree.

Contents

Executive summary

The renderer is an **ordered, single-stream command submission system**. Every normal frame rebuilds a temporary Scene list, asks each drawable to append RenderCommand tokens, submits those tokens to IBackend, and finally swaps the GLFW window. The production backend owns OpenGL objects in handle-indexed registries; GLDevice performs the actual calls. The same command-generation path can be exercised headlessly with MockBackend.

The main Canvas is not a per-cell mesh. Logical cellular-automata bytes remain on the CPU. A CPU palette produces target RGB values, presentation colors fade toward those targets, a dirty RGB subrectangle is uploaded, and one indexed quad samples that texture. UI is similarly simple: GLString and CommandLine build CPU quads, upload dynamic vertex data, and issue one indexed draw per drawable or batch.

- **Best architectural property:** draw intent is testable without a window or GPU because drawables emit inspectable tokens.
- **Best performance property:** the Canvas uses one draw call and skips texture uploads on visually idle frames.
- **Most important correctness risk:** the 2,048-command queue silently drops overflow while drawables can clear their pending-update flags immediately after enqueue attempts.
- **Most important boundary limitation:** IBackend is injectable, but production construction and string-named uniforms remain OpenGL-shaped.
- **Most important scope observation:** render-pass, framebuffer, uniform-buffer, batched-draw, and instancing concepts exist mainly as scaffolding and should not be described as implemented features.

BOTTOM LINE

Keep the renderer small. Improve command acceptance reporting, viewport correctness, and state ownership before considering any generalized render graph or second production graphics API.

1. Scope, method, and confidence

1.1 Scope

This report covers the active renderer from application composition through OpenGL execution. It includes Scene ordering, Drawable behavior, Renderer APIs, RenderCommand layout, CommandQueue semantics, IBackend, GLBackend registries, GLDevice interpretation, Canvas presentation, text and console batching, the token proof path, MockBackend tests, and the boundaries between active code and dormant scaffolding.

1.2 Inspection method

1. Read the repository bootstrap, README, and canonical architecture consensus.
2. Inspected the live implementation files; source code was treated as authoritative when prose disagreed.
3. Traced construction, update, render, submission, resource lookup, OpenGL execution, and shutdown paths.
4. Compared production drawables with headless MockBackend and end-to-end renderer tests.
5. Executed the existing build/Release/CSimTests.exe and observed 0 failures. The executable was not rebuilt during this report generation.
6. Did not launch the graphical application, inspect pixels, or validate real GPU and window behavior.

1.3 Confidence boundaries

Area	Confidence	Reason
Control flow and ownership	High	Directly traced through CellMain, Illumo, Renderer, GLBackend, and shutdown code.
Token order and payloads	High	Direct code inspection plus MockBackend end-to-end coverage.
Canvas CPU presentation	High	Live Canvas code, shaders, and focused tests agree.
OpenGL call mapping	High	GLDevice and resource wrapper implementations inspected directly.
Actual visual output	Moderate	Shaders and GL code inspected, but no live window or screenshot validation.
High-DPI and platform parity	Moderate to low	Windows is production; framebuffer behavior and non-Windows paths were not exercised.

2. Renderer position in the application

Renderer behavior begins above the Rendering package. CellMain owns product composition, Illumo owns long-lived services, and modules decide what appears. This keeps the engine host independent of Game classes while still giving modules a narrow service bag containing the renderer, camera, window, and Scene.

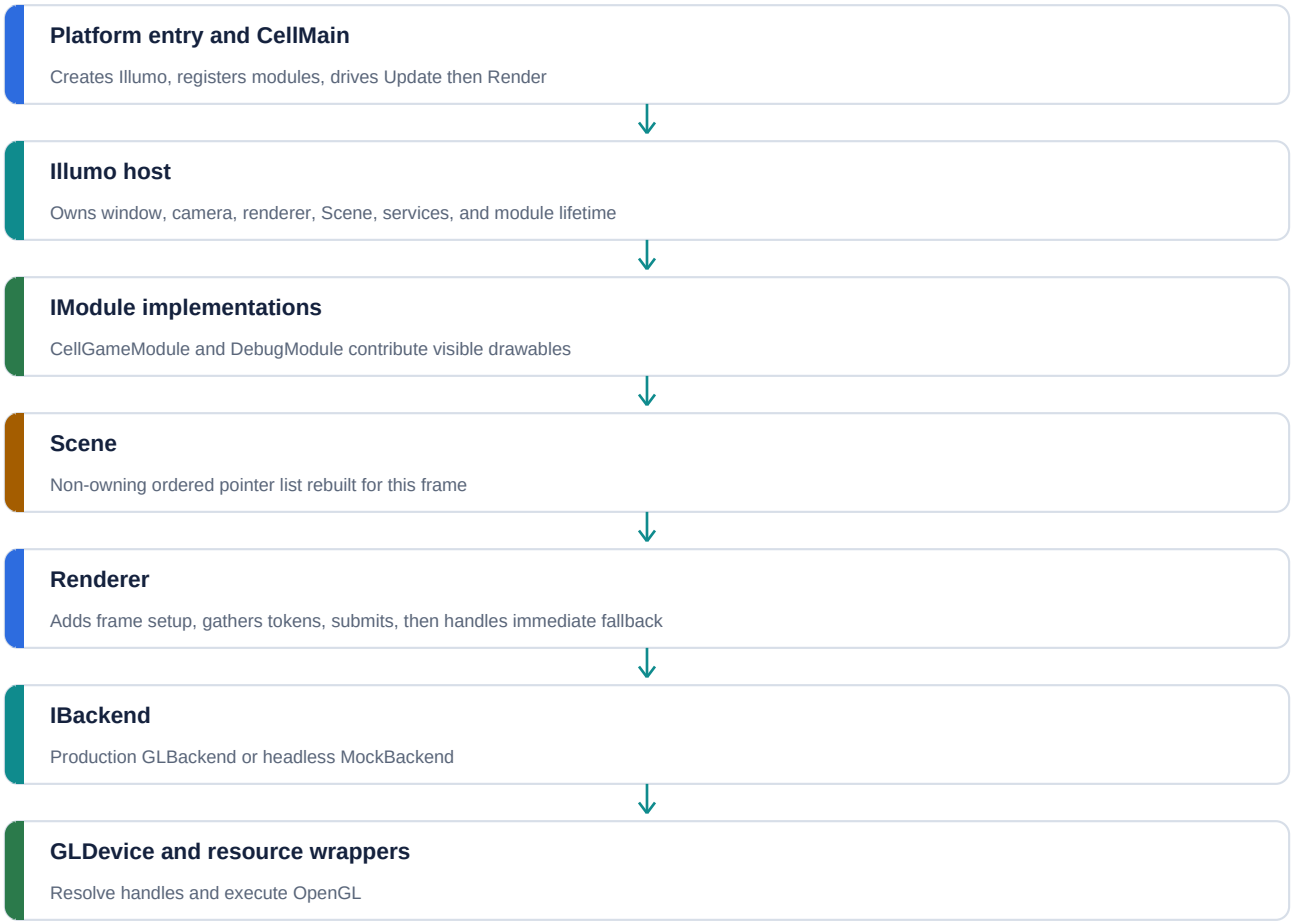


Figure 1. Active renderer control flow from product loop to graphics API.

`Sources: CSim/Source/App/CellMain.cpp:10-44; Engine/Illumo.cpp:101-181; Rendering/Renderer.h:367-415`

2.1 Product composition

CellMain creates Illumo, initializes services, registers CellGameModule, optionally registers DebugModule in Debug builds, and then executes a tight loop of Update followed by Render. Renderer behavior therefore observes a complete simulation update for the current frame; it does not draw midway through a ruleset generation.

```
while (!illumo->ShouldClose())
{
    double currentTime = glfwGetTime();
    double dt = currentTime - lastTime;
    lastTime = currentTime;

    illumo->Update(dt);
    illumo->Render();
}
```

CSim/Source/App/CellMain.cpp:31-39

2.2 Module contribution order

Illumo clears Scene::drawables and calls DispatchDrawables on modules in registration order. CellGameModule contributes the Canvas first and then the optional mode splash. DebugModule contributes the console and then the FPS label. Because Renderer walks the vector linearly, this creates painter-style overlay ordering without a pass system.

Order	Producer	Drawable	Typical pipeline
1	CellGameModule	Canvas	Depth off, blending off, textured indexed quad
2	CellGameModule	SplashText	Depth off, alpha blending on, dynamic text geometry
3	DebugModule	CommandLine	Depth off, alpha blending on, single packed UI batch
4	DebugModule	GLString FPS label	Depth off, alpha blending on, cached text geometry

Sources: Game/CellGameModule.cpp:492-500; Engine/DebugModule.cpp:173-184

3. Ownership and resource lifetime

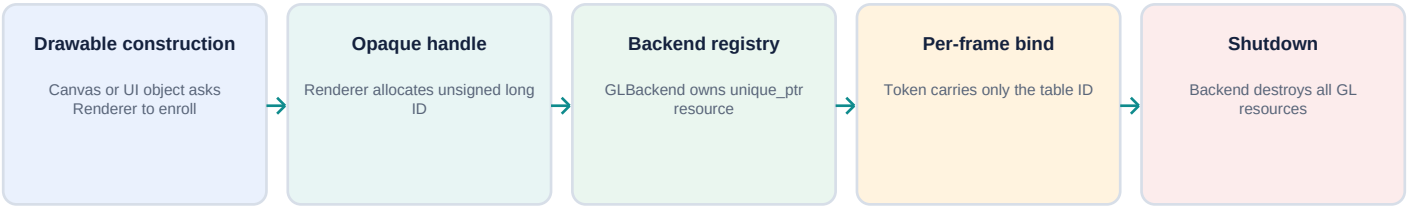
3.1 Long-lived service ownership

Illumo owns EnvVars, Camera, RenderWindow, Renderer, AssetManager, CommandRegistry, CommandLine, InputManager, Scene, and modules through unique_ptr. IllumoContext contains non-owning raw pointers into those services. The renderer is constructed after the OpenGL window and context exist, and it is destroyed before the window because member destruction runs in reverse declaration order.

Owner	Owned object	Renderer significance
Illumo	RenderWindow	Creates GLFW window and current OpenGL context before GLBackend construction.
Illumo	Camera	Supplies world-to-clip matrix for Canvas.
Illumo	Renderer	Owns production GLBackend; exposes enrollment and token helpers.
Illumo	Scene	Per-frame non-owning drawable list.
Renderer	GLBackend	Owned in production; optionally injected and non-owned in tests.
GLBackend	GLDevice and CommandQueue	Created in backend constructor; destroyed by Shutdown.
GLBackend	GLMesh, GLShaderProgram, GLTexture	Owned in per-resource-type unordered_map registries.
Modules	Canvas, SplashText, FPS label	CPU objects retain staging memory used by update tokens.

Sources: Engine/Illumo.h:43-56; Engine/Illumo.cpp:101-120; Rendering/Renderer.h:68-102; OpenGL/GLBackend.h:15-25

3.2 Enrollment lifecycle



Current lifetime policy: create synchronously, retain until renderer shutdown, never recycle handles.

Figure 2. Production resource lifetime and handle flow.

Renderer::allocateHandle returns monotonically increasing unsigned long values beginning at 1. enrollMesh, enrollShader, and enrollTexture immediately call the backend Create methods. GLBackend uses the caller-provided ID as the key in a registry and constructs the OpenGL wrapper synchronously. Per-frame commands carry only the table ID, not a raw OpenGL name.

- Handles are untyped; a shader handle, mesh handle, and texture handle share the same C++ representation.
- Registries are separated by resource type, so equal numeric values can exist in different registries without collision.
- Renderer-owned handles are never recycled.
- Resources are destroyed only during GLBackend::Shutdown; normal drawables do not free their backend entries when their CPU object is deleted.
- The test constructor can inject MockBackend and set takeOwnership=false, preventing Renderer from shutting down a stack-allocated mock.

3.3 AssetManager's current role

AssetManager is constructed and exposed through IllumoContext, but the active Canvas, CommandLine, GLString, and token-proof paths enroll directly through Renderer. AssetManager maintains counters and placeholder shared_ptr registries, yet its load methods forward resources into Renderer without populating those shared_ptr registries. Consequently, GLBackend is the authoritative owner in current production rendering. File-based mesh creation is also unimplemented in GLBackend.

`Sources: Rendering/AssetManager.h:14-132; OpenGL/GLBackend.cpp:132-137`

4. Exact frame lifecycle

4.1 Normal production frame

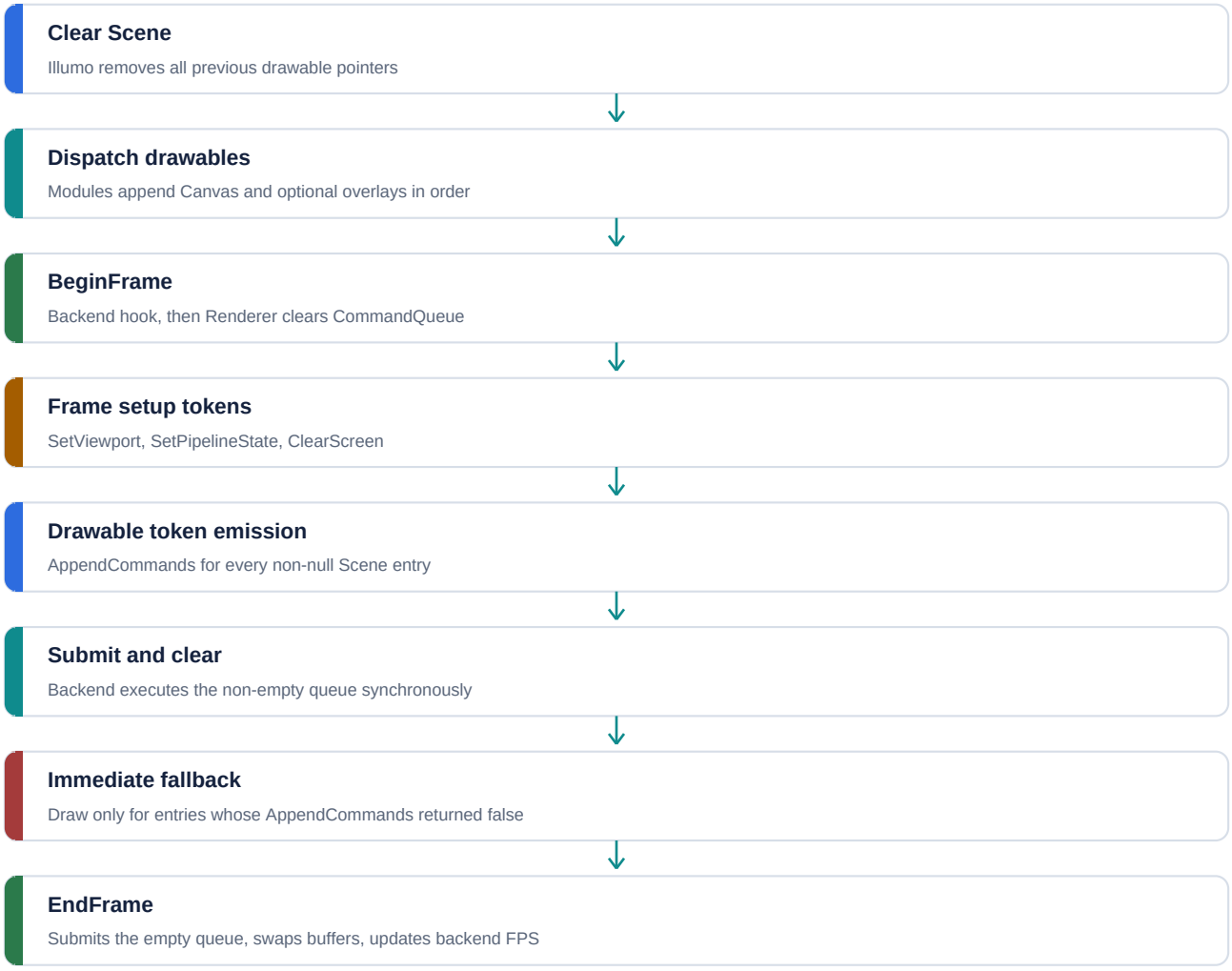


Figure 3. Normal-frame sequencing and submission boundaries.

4.2 BeginFrame

Renderer::BeginFrame calls IBackend::BeginFrame and then clears the backend command queue. GLBackend::BeginFrame is currently empty. This is a lifecycle seam rather than an active synchronization or fence stage.

`Sources: Rendering/Renderer.h:187-191; OpenGL/GLBackend.cpp:41-43`

4.3 RenderScene frame setup

Before visiting the Scene, Renderer emits three tokens. The viewport covers the current logical window dimensions. The default pipeline enables depth testing, disables blending and face culling, and selects triangles. ClearScreen sets the historical dark gray clear color and clears both color and depth. Individual drawables then replace relevant pipeline state before drawing.

```
  SetViewport(0, 0, windowWidth, windowHeight)
  SetPipelineState(depth=true, blend=false, cull=false, primitives=Triangles)
  ClearScreen(0.1, 0.1, 0.1, 1.0)

  for drawable in Scene order:
      if drawable.AppendCommands(Renderer) == false:
          immediateList.push_back(drawable)

  SubmitCommandQueue()
  ClearCommandQueue()
  for drawable in immediateList:
      drawable.Draw()
```

Rendering/Renderer.h:367-415

4.4 EndFrame and swap

Renderer::EndFrame unconditionally submits the queue again and then calls the backend EndFrame hook. After a normal RenderScene call the queue is empty, so this is an empty second submission. GLBackend::EndFrame swaps the GLFW buffers and calculates an approximate FPS value once per second. The debug FPS label does not use this backend FPS field; DebugModule maintains its own counter from application dt.

Sources: Rendering/Renderer.h:193-199; OpenGL/GLBackend.cpp:45-62; Engine/DebugModule.cpp:43-73

4.5 Token proof mode

When UseTokenProof=1, Illumo bypasses Scene construction and calls Renderer::RenderProofQuad. The renderer lazily enrolls a quad, shader pair, and 2x2 checkerboard texture; clears its queue; emits a complete token sequence; submits; and clears again before EndFrame. This development path proves that a frame can be produced entirely through IBackend and RenderCommand without Drawable involvement.

Sources: Engine/Illumo.cpp:152-165; Rendering/Renderer.h:418-499

5. Scene and Drawable semantics

5.1 Scene is a frame list, not a world model

Scene owns no drawables, resources, entities, transforms, or hierarchy. It stores a window pointer, an active camera pointer, and an ordered vector of DrawableBase pointers. ClearDrawables invalidates only the contribution list; it does not destroy objects. This design makes Scene cheap to rebuild every frame and keeps product composition in modules.

- No scene graph traversal.
- No entity-to-renderable registry.
- No render-layer enum or explicit pass assignment.
- No sorting by material, shader, depth, or transparency.
- No ownership transfer when a drawable is added.

Rendering/Scene.h:7-34

5.2 Drawable contract

DrawableBase exposes two paths. AppendCommands is the preferred token path and returns true when it handled the drawable for the frame. Draw is the legacy immediate path. The CRTP Drawable template implements Draw by checking visibility and forwarding to Derived::DrawImpl. Production drawables also perform their own visibility checks inside AppendCommands.

AppendCommands result	Renderer behavior	Current use
true	Do not call Draw; submitted tokens represent the drawable or it intentionally emitted nothing.	All production drawables when operational or invisible.
false	Place object on immediateList and call Draw after token submission.	Headless test stub or future unmigrated drawable.

ORDERING CAVEAT

All token drawables are submitted before every immediate fallback object. Therefore a legacy object is not interleaved at its Scene position; it is always drawn after the complete token stream.

Sources: Rendering/Drawable.h:13-48; Rendering/Renderer.h:388-413

6. Renderer API and command construction

6.1 Renderer as a thin command facade

Renderer does not retain a rich render-world model. Its main job is to convert typed helper calls into RenderCommand structs and forward those structs to IBackend::PushToCommandQueue. Helpers fill only the union payload relevant to the selected CommandType. Uniform names are copied into fixed arrays to keep commands self-contained and trivially copyable.

Renderer helper	Token	Payload
pushViewport	SetViewport	x, y, width, height
pushPipelineState	SetPipelineState	Complete PipelineState value
pushClearScreen	ClearScreen	RGBA clear values
pushSetShader	SetShader	Opaque handle
pushSetMesh	SetMesh	Opaque handle
pushSetTexture	SetTexture	Opaque handle and texture slot
pushUniformInt	SetUniformInt	31-character name plus int
pushUniformFloat	SetUniformFloat	31-character name plus float
pushUniformVec2	SetUniformVec2	31-character name plus x/y
pushUniformMat4	SetUniformMat4	31-character name plus copied 16 floats
pushUpdateTexture	UpdateTexture	Handle, rect, channel count, row stride, raw pointer
pushUpdateBuffer	UpdateBuffer	Mesh handle, byte offset, byte count, raw pointer
pushDrawIndexed	DrawIndexed	Element count and first index

Rendering/Renderer.h:210-361

6.2 String uniform rules

Uniform commands store names in char[32]. copyUniformName always null-terminates, but names longer than 31 bytes are truncated. GLDevice resolves names against whichever shader program was most recently selected and caches locations using a program-ID:name key. A SetUniform token issued before SetShader produces no update because _activeProgram is zero. Missing or optimized-out uniforms are ignored when glGetUniformLocation returns -1.

Sources: Rendering/Renderer.h:45-64, 278-316; OpenGL/GLDevice.h:67-82; OpenGL/GLDevice.cpp:205-244

6.3 Pointer payload lifetime

UpdateTexture and UpdateBuffer do not copy their source bytes into the command. They copy only a raw pointer. The pointed-to data must remain valid and unchanged until SubmitCommandQueue returns. Current production sources satisfy this because Canvas owns texCanvasBuffer for its entire lifetime, GLString owns its fixed vertex array, CommandLine owns uiVerts, and submission happens immediately after all AppendCommands calls.

DESIGN CONSTRAINT

The current payload model is synchronous by construction. Deferring execution, building commands on worker threads, or retaining frames in flight would require owned staging memory, frame arenas, or explicit upload copies.

Rendering/RenderCommand.h:4-5, 111-130

7. RenderCommand and CommandQueue

7.1 Tagged-union command model

RenderCommand contains a CommandType, a PipelineState value, and a C-style union of payload structs. This avoids virtual command objects, heap allocation per command, std::variant visitation, and byte-stream decoding. The queue stores commands by value. The tradeoff is that type-to-payload correctness is conventional: the producer and consumer must agree which union member is valid for each CommandType.

Category	Declared commands	Production status
Pipeline	SetPipelineState, SetViewport, SetScissor	Pipeline and viewport active; scissor interpreter exists but no production emitter.
Bindings	SetShader, SetMesh, SetVertexBuffer, SetIndexBuffer, SetTexture, SetUniformBuffer	Shader, mesh, texture active; direct buffer binds are legacy; uniform buffer ignored.
Uniforms	Int, Float, Vec2, Mat4	All interpreted; Int, Vec2, Mat4 currently emitted by production drawables.
Updates	UpdateTexture, UpdateBuffer	Active for Canvas and dynamic UI geometry.
Clears	ClearScreen, ClearDepthBuffer, ClearColorBuffer, ClearStencilBuffer, ClearAll	ClearScreen active; others interpreted but not normally emitted.
Draw	Draw, DrawIndexed, DrawInstanced, DrawBatched	DrawIndexed active; Draw and DrawInstanced interpreted; DrawBatched ignored.
Passes	BeginRenderPass, EndRenderPass	Declared and ignored.

Sources: Rendering/RenderCommand.h:7-146; OpenGL/GLDevice.cpp:94-347

7.2 Fixed-capacity queue

CommandQueue allocates an ArrayQueue with capacity 2,048. It uses the underlying array as indexed storage and tracks a separate m_CommandCount. Reset sets only the count to zero. Submit copies the command when the count is below the limit and otherwise returns silently. No boolean, status enum, assertion, warning, telemetry counter, or high-water mark reveals rejection.

```
void Submit(RenderCommand command) {
    if (m_CommandCount < MAX_DRAW_COMMANDS) {
        (*commandQueue)[m_CommandCount] = command;
        m_CommandCount++;
    }
}
```

Rendering/CommandQueue.h:7-29

CRITICAL CONSEQUENCE

Canvas and GLString clear textureUploadPending or gpuUploadPending after asking Renderer to enqueue an update. If the queue is already full, the command can be dropped while the CPU object believes the upload succeeded. This turns capacity pressure into silent visual staleness.

8. Backend abstraction

8.1 IBackend contract

IBackend separates resource creation and command submission from the drawable layer. It exposes lifecycle hooks, queue operations, FPS access, mesh/shader/texture creation, and a placeholder descriptor-set function. This is sufficient to substitute MockBackend for GLBackend in headless tests.

Interface group	Operations	Current meaning
Lifecycle	Initialize, Shutdown, BeginFrame, EndFrame	GL setup and shutdown; BeginFrame is currently empty; EndFrame swaps.
Submission	PushToCommandQueue, SubmitCommandQueue, ClearCommandQueue	Backend owns queue and execution boundary.

Interface group	Operations	Current meaning
Metrics	getFPS	GLBackend counts swaps per elapsed second.
Resources	CreateMesh, CreateShaderProgram, CreateTexture	Synchronous registry insertion and OpenGL creation.
Future-facing	CreateDescriptorSet	Returns 0 in GLBackend and MockBackend records only a placeholder.

Rendering/IBackend.h:8-33

8.2 What is portable and what is OpenGL-shaped

Portable boundary	OpenGL-shaped boundary
Opaque resource handles rather than raw GLuint values	Renderer production constructor directly includes and creates GLBackend
Pure PipelineState enums for blend, cull, winding, and primitives	Uniforms are addressed by shader source names
Update and draw intent represented as tokens	Mesh creation vocabulary assumes vertex/index buffers and fixed layouts
MockBackend can record commands without a context	Texture slots and shader-program sequencing mirror OpenGL usage
Window swap is abstracted by IRenderWindow	GLFW types remain exposed for input callbacks and production window setup

This is a reasonable compromise for one real backend. A second production API would force decisions about typed bindings, resource-state transitions, pipeline objects, synchronization, command data ownership, and error reporting. Until then, expanding the abstraction would add ceremony without a demonstrated product need.

9. GLBackend resource registries

9.1 Construction and initialization

GLBackend allocates GLDevice and CommandQueue, stores the window pointer, and immediately calls Initialize. Initialize runs glewInit and logs the OpenGL version. RenderWindow already performs GLEW initialization after creating the context, so the current startup calls glewInit twice. The second call is usually harmless but redundant.

Sources: Rendering/RenderWindow.cpp:32-89; OpenGL/GLBackend.cpp:18-39

9.2 Registry lookup

Three unordered_map registries store unique_ptr values keyed by unsigned long handles: meshes, programs, and textures. SubmitCommandQueue packages const pointers to those registries into GLResourceTables and asks GLDevice to execute the queue. GLDevice resolves each handle at the command that needs it. Unknown shader, mesh, or texture handles produce warnings and execution continues.

Sources: OpenGL/GLBackend.h:19-25; OpenGL/GLBackend.cpp:64-71; OpenGL/GLDevice.h:14-22, 84-120

9.3 Shutdown

Shutdown iterates each registry, calls the resource's explicit Destroy method, clears the maps, and deletes GLDevice and CommandQueue. GLBackend's destructor itself is empty, so correct cleanup depends on Renderer invoking Shutdown before deleting an owned backend. Renderer does so in its destructor. Repeated Illumo::Shutdown calls clear modules twice safely, while backend shutdown remains tied to renderer destruction.

Sources: OpenGL/GLBackend.cpp:73-119; Rendering/Renderer.h:95-102; Engine/Illumo.cpp:183-194

10. GLDevice command execution

10.1 Submission-local binding cache

At the beginning of every ExecuteCommandQueue call, GLDevice clears its cached program, VAO, eight texture IDs, viewport, and active program. This intentionally avoids assuming that a previous submission left bindings intact. Within the current queue, redundant binds and viewport changes are skipped.

- SetShader resolves the handle, calls glUseProgram only when the GLuint differs, and updates _activeProgram for later uniforms.
- SetMesh resolves the handle and binds the VAO only when its ID differs.
- SetTexture resolves the handle, activates the requested slot, and avoids a repeated bind when the slot already tracks that texture.
- SetViewport compares x, y, width, and height before calling glViewport.
- Uniform locations remain cached across submissions by OpenGL program ID and uniform name.

`OpenGL/GLDevice.cpp:83-205`

10.2 Pipeline state application

ApplyPipelineState compares requested state with _currentGLState and calls OpenGL only for differences. It controls depth testing, blending, face culling, winding, wireframe mode, and primitive topology. Blend enabling always applies glBlendFunc because OpenGL's default ONE/ZERO factors do not match the CPU-side SrcAlpha/OneMinusSrcAlpha defaults.

Pipeline field	OpenGL effect	Current users
depthTestEnabled	glEnable/glDisable(GL_DEPTH_TEST)	Frame default true; Canvas and UI override false
blendEnabled	glEnable/glDisable(GL_BLEND)	UI true; Canvas false
blendSrc/blendDst	glBlendFunc	UI uses SrcAlpha / OneMinusSrcAlpha
faceCullingEnabled	glEnable/glDisable(GL_CULL_FACE)	Disabled by all active drawables
cullFace/frontFace	glCullFace/glFrontFace	Only meaningful if culling is enabled
wireframe	glPolygonMode	Not enabled by production drawables
primitives	Selects GL_POINTS, GL_LINES, or GL_TRIANGLES for draw	Triangles throughout active production path

`OpenGL/GLDevice.cpp:4-81`

10.3 Draw execution

DrawIndexed converts firstIndex into a byte offset by multiplying by sizeof(unsigned int), then calls glDrawElements with GL_UNSIGNED_INT. The currently bound shader, VAO, element buffer, textures, uniforms, and PipelineState determine the result. GLDevice does not verify that elementCount fits the mesh's uploaded index buffer.

`OpenGL/GLDevice.cpp:306-314`

10.4 Partial state ownership

Binding and viewport caches reset per submission, but _currentGLState does not. If code outside GLDevice changes depth, blend, culling, winding, or wireframe state, the CPU cache can become inaccurate. Production drawables are token-only and window-level OpenGL calls are mostly bootstrap and resize operations, so this risk is currently contained. It becomes material if legacy immediate rendering or new raw GL code is added.

11. Canvas presentation pipeline

Canvas intentionally combines logical grid storage, palette-driven presentation, dirty tracking, CPU fade buffers, GPU resource enrollment, and token emission. That is broad responsibility, but the current architecture treats it as an intentional product-scale monolith rather than a premature split into LifeGrid and CanvasView.

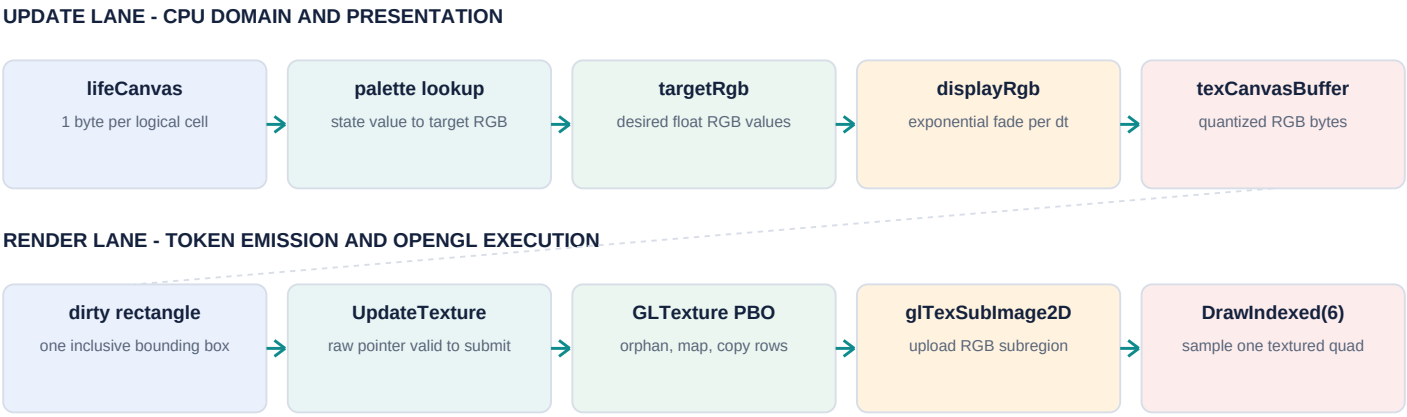


Figure 4. Canvas domain-to-presentation pipeline.

11.1 Logical state

lifeCanvas contains width * height unsigned bytes. Binary rules encode 0 as alive and 1 as dead; multistate rules may use additional values. setCanvasPixel updates this logical byte and expands cellsDirtyRect only when the value actually changes. Simulation and editing never write OpenGL resources directly.

`Game/Canvas.h:97-140; Game/Canvas.cpp:94-137`

11.2 Palette and target colors

paletteRgb contains 256 RGB triplets. rebuildPalette asks the active RuleSet to evaluate every possible byte value, then marks the full Canvas logically dirty because existing state values may now map to different colors. rebuildTargetsFromLife scans only cellsDirtyRect when valid and writes normalized float targets into targetRgb. Changed targets activate fading and expand fadeDirtyRect.

`Game/Canvas.cpp:39-92, 235-326`

11.3 Exponential fade

tickVisual computes alpha = 1 - exp(-fadeSpeed * dt). Each affected displayRgb channel moves toward its target by diff * alpha. An epsilon of 0.002 snaps nearly completed channels exactly to their targets. Each cell is quantized to byte RGB through texCanvasBuffer; actual byte changes expand uploadDirtyRect. fadeSpeed <= 0 selects the immediate snap path.

- Visual interpolation is independent of simulation state and generation timing.
- Idle frames return immediately when fadeActive is false.
- The fade scans one bounding rectangle; it does not maintain a sparse set of individual cells.
- Byte comparison prevents texture uploads for float changes too small to alter the displayed 8-bit value.

`Game/Canvas.cpp:328-480`

11.4 Canvas geometry and shaders

The Canvas mesh contains four vertices and six unsigned indices. World dimensions equal cell count multiplied by a fixed 16-pixel cell size. The vertex layout contains position, an unused color triplet, and texture coordinates. The vertex shader reads position at location 0 and UV at location 2, applies uMVP, and forwards UV. The fragment shader samples uDisplayTexture and returns that RGB value.

Resource	Representation	Behavior
Mesh	4 vertices, 6 indices	One quad spans the entire logical grid in world space.
Vertex layout	pos3 | color3 | uv2, 32-byte stride	Color values are present in the data but ignored by the Canvas shader.
Texture	GL_RGB8, width x height	One texel per cell, nearest filtering, clamp-to-edge.
Vertex shader	canvas_vertex.glsl	uMVP transforms world-space quad positions.
Fragment shader	canvas_frag.glsl	Direct RGB texture sample; no palette lookup or fade on GPU.

Sources: Game/Canvas.cpp:94-174, 555-574; Shader/canvas_vertex.glsl:1-14; Shader/canvas_frag.glsl:1-12

11.5 Canvas token sequence

```
if textureUploadPending:
    UpdateTexture(displayTextureHandle, dirtyRect, RGB, texCanvasBufferPointer)

SetPipelineState(depth=false, blend=false, cull=false, Triangles)
SetShader(canvasShader)
SetMesh(canvasQuad)
SetTexture(displayTexture, slot=0)
SetUniformMat4("uMVP", cameraMvp)
SetUniformInt("uDisplayTexture", 0)
DrawIndexed(elementCount=6, firstIndex=0)
```

Even when the display texture is unchanged, Canvas still emits state, bind, uniform, and draw tokens. Only the UpdateTexture token is conditional. Because the Canvas occupies a single quad, this constant per-frame command cost is small.

Game/Canvas.cpp:482-577

12. Texture upload path

12.1 Dirty rectangle formation

DirtyRect stores inclusive min and max coordinates. Multiple writes expand a single axis-aligned bounding box. Canvas uses separate rectangles for logical changes, active fade, and pending texture upload. AppendCommands converts uploadDirtyRect into x, y, width, and height, points into the appropriate pixel within texCanvasBuffer, and sets srcRowStride to the full Canvas width.

Sources: Game/Canvas.h:12-68; Game/Canvas.cpp:495-553

12.2 PBO ping-pong algorithm

1. Normalize the requested channel count and derive OpenGL format.
2. Ensure two pixel-unpack buffers exist with capacity for the entire texture.
3. Select the alternate PBO and orphan its previous storage with glBufferData(..., nullptr, GL_STREAM_DRAW).
4. Map the PBO for writing. If mapping fails, unbind the PBO and upload directly from client memory.
5. Copy each dirty source row into the byte offset matching that row's full-texture location in the PBO.
6. Unmap, bind the texture, configure unpack alignment and row length, and call glTexSubImage2D using a PBO byte offset.
7. Restore GL_UNPACK_ROW_LENGTH when it was changed and unbind the PBO.

Rendering/OpenGL/GLTexture.h:75-159

12.3 Cost characteristics

Cost	Current behavior	Implication
PBO allocation size	Full texture bytes for both PBOs	Small dirty updates still retain two full-size upload buffers.
PBO orphan size	Full texture size per update	Driver storage is refreshed at full capacity even for a tiny region.
CPU memcpy	Only dirty rows and width	Sparse edits reduce actual copied bytes.
GPU texture upload	Dirty rectangle only	Idle and localized edits avoid full glTexSubImage2D transfers.
Dirty representation	One AABB	Two distant cells may create a large upload region.

13. Camera, viewport, and coordinate systems

13.1 Canvas world coordinates

Each cell occupies 16 x 16 world units. Camera initializes at the Canvas center and provides an orthographic projection based on WinX and WinY environment variables. View construction translates to screen center, rotates, scales by zoom, and translates relative to camera position. Canvas sends projection * view as uMVP.

Sources: Rendering/Camera.cpp:6-31, 70-111; Game/Canvas.cpp:116-126, 569-573

13.2 Screen-space UI

GLString and CommandLine do not use Camera. Their shaders receive u_resolution and convert pixel-space x/y into normalized device coordinates. The y coordinate is inverted so UI origins follow top-left screen conventions. This separation lets camera pan and zoom affect the Canvas without moving the console or FPS overlay.

Sources: Rendering/GLString.cpp:12-34; Services/CommandLine.cpp:14-35

13.3 High-DPI viewport concern

RenderWindow sets the initial and resize viewport using glfwGetFramebufferSize, which is appropriate for high-DPI displays. Renderer::RenderScene later emits SetViewport using getWindowDimensions, which stores logical window dimensions. On systems where framebuffer pixels differ from logical window units, the per-frame token can overwrite the correct framebuffer viewport with a smaller logical viewport. This is an inference from the two code paths and should be verified on a scaled display.

Sources: Rendering/RenderWindow.cpp:72-76, 119-133; Rendering/Renderer.h:375-377

14. UI rendering

14.1 GLString resource model

GLString reserves CPU storage for up to 2,000 stb_easy_font quads. Enrollment creates a dynamic VBO sized for 8,000 VertexData records and a static index buffer containing six indices per quad. Its vertex format is float position plus four normalized unsigned-byte color channels. The inline shader transforms screen-space geometry and passes vertex color directly to the fragment stage.

Rendering/GLString.cpp:8-129; Rendering/OpenGL/GLMesh.h:112-178

14.2 GLString caching

Changing content, RGB, or alpha marks geometry dirty and upload pending. Position and scale are uniforms, so moving text or changing its nominal size does not rebuild geometry. On AppendCommands, dirty geometry is regenerated once. UpdateBuffer is emitted only while gpuUploadPending is true; later frames reuse the VBO and issue only state, bind, uniform, and draw tokens.

Property change	CPU geometry rebuild	VBO upload	Uniform-only
Content	Yes	Yes	No
R/G/B/A	Yes	Yes	No

Property change	CPU geometry rebuild	VBO upload	Uniform-only
X/Y position	No	No	Yes
Size	No	No	Yes, u_scale
Visibility false	No	No	No draw emitted
Empty content	No useful geometry	No	No draw emitted

Rendering/GLString.cpp:131-296

14.3 CommandLine batching

CommandLine builds one combined vertex array containing panel chrome, separator, scrollbar, history text, and the editable prompt. The mesh's static index buffer supports 2,000 quads. After animation and layout calculations, AppendCommands emits one UpdateBuffer and one DrawIndexed for the entire visible console. This is the renderer's principal batching optimization.

- The CPU batch capacity is 12,000 vertices, while drawing is clamped to 2,000 quads to match the index buffer.
- The console is not dispatched when fully closed and no close animation remains.
- Because panel alpha is baked into vertex colors, standard SrcAlpha/OneMinusSrcAlpha blending is required.
- The console currently rebuilds and uploads its visible batch each drawn frame rather than caching static history geometry separately.

Sources: Services/CommandLine.h:65-76; Services/CommandLine.cpp:487-640; Engine/DebugModule.cpp:173-184

14.4 SplashText

SplashText derives from GLString. While visible, it delegates token generation and then advances a linear 1.5-second fade based on wall-clock time. Alpha changes mark GLString geometry dirty because color is stored per vertex, so an actively fading splash can upload its text VBO every rendered frame until hidden.

Sources: Rendering/SplashText.h:5-47; Rendering/SplashText.cpp:3-42

15. OpenGL resource wrappers

15.1 GLMesh

GLMesh creates a VAO and VBO, optionally creates an EBO, configures one of two fixed vertex layouts, and retains limited CPU copies for static float meshes. Dynamic meshes allocate VBO capacity with GL_DYNAMIC_DRAW and update subranges through glBufferSubData. Index buffers are uploaded as unsigned int data and remain static.

Layout	Stride	Attributes	Users
Pos3Color3Uv2	32 bytes	location 0: pos3 float; location 2: uv2 float	Canvas and token-proof quad
Pos3Color4U8	16 bytes	location 0: pos3 float; location 1: normalized color4 byte	GLString and CommandLine

Rendering/OpenGL/GLMesh.h:7-199

15.2 GLShaderProgram

GLShaderProgram accepts file paths or inline strings, compiles vertex and fragment shaders, links a program, prints compile/link failures to stderr, deletes intermediate shader objects, and retains the program until explicit Destroy. A failed shader compile returns shader ID 0, but the link path still attempts to attach and link it. Resource enrollment does not currently return a rich failure result.

Rendering/OpenGL/GLShaderProgram.h:10-104

15.3 GLTexture

GLTexture supports R8, RGB8, and RGBA8 internal formats. Memory-backed Canvas texture creation uses RGB8. File-backed textures are loaded through stb_image and forced to RGBA. Filtering is nearest-neighbor and wrapping clamps to edges. R8 textures receive a swizzle that copies red into RGB, but R8 is not the current Canvas presentation path.

Rendering/OpenGL/GLTexture.h:9-278

16. Headless testing and observability

16.1 MockBackend behavior

MockBackend implements IBackend without touching OpenGL. It records Create calls, stores queued commands in the same CommandQueue type, snapshots submissions, preserves the most recent non-empty submission across the empty EndFrame submit, and exposes inspection helpers for counts, command order, payloads, and resource records.

Rendering/Mock/MockBackend.h:1-250

16.2 Test coverage matrix

Suite	Renderer coverage	Not covered
TestMockBackend	Lifecycle, Create records, queue snapshots, proof-like sequence, texture pointer payload	Actual Renderer orchestration and OpenGL
TestRendererE2E	Injected backend, Scene setup prefix, token drawable, immediate fallback, Canvas RGB dirty update, proof path	GLDevice calls and real context
TestCanvasDomain	Palette, fade, logical dirty flags, upload dirty rectangle, resource enrollment	PBO implementation and shader output
TestUITokens	Console batching, visibility, blending, GLString update caching, combined Scene draws	Pixel-perfect UI and GPU buffer capacity validation

Sources: Tests/TestMockBackend.cpp; TestRendererE2E.cpp; TestCanvasDomain.cpp; TestUITokens.cpp

16.3 Validation result for this report

HEADLESS RESULT

The existing build/Release/CSimTests.exe was executed during the source inspection and reported ALL PASSED with 0 failures across MockBackend, Renderer E2E, rulesets, CellContext, Canvas domain, and UI token suites. No rebuild or live OpenGL smoke test was performed.

16.4 Observability gaps

- No queue overflow counter or high-water telemetry.
- No per-frame command breakdown exposed in production.
- No resource-registry size or memory statistics surfaced to DebugModule.
- No automated real-OpenGL integration test or framebuffer screenshot comparison.
- No GL error checks around individual command execution.
- Shader creation reports errors to stderr but does not propagate structured failure to the caller.

17. Performance characteristics

17.1 Expected normal-frame command shape

A minimal frame with only an unchanged Canvas normally contains three frame-setup commands and seven Canvas commands: pipeline, shader, mesh, texture, MVP uniform, sampler uniform, and indexed draw. A dirty Canvas adds one UpdateTexture. Each visible GLString adds approximately seven or eight commands depending on whether geometry must upload. The visible console adds six commands: pipeline, shader, mesh, two vec2 uniforms, update, and draw - seven in total when counted individually.

Scenario	Approximate commands	Draw calls	Uploads
Idle Canvas only	10	1	0

Scenario	Approximate commands	Draw calls	Uploads
Dirty Canvas only	11	1	1 texture subregion
Canvas + unchanged FPS	17	2	0 after FPS cache warm-up
Canvas + open console	17-18	2	1 console VBO plus optional Canvas texture
Canvas + splash + console + FPS	Approximately 31-34	4	Console VBO, fading splash VBO, optional Canvas and FPS updates

These counts remain far below the 2,048-command capacity in current production. The overflow hazard matters because the policy is silent and future features could increase command volume, not because today's normal frame is near the limit.

17.2 Draw-call profile

- Canvas: one indexed draw regardless of grid dimensions.
- CommandLine: one indexed draw for its complete visible UI batch.
- Each GLString or SplashText: one indexed draw.
- No cross-drawable material sorting or multi-draw aggregation.
- No instancing is needed for cells because the Canvas is a texture, not a cell mesh.

17.3 CPU and memory profile

Canvas presentation stores one byte of logical state, six float channels across displayRgb and targetRgb, and three display bytes per cell. At four-byte floats this is approximately 28 bytes per cell before allocator overhead and unrelated simulation scratch buffers. The default 80 x 60 grid is small. Large grids would make CPU fade scanning, staging memory, full-size PBO capacity, and the simulation's full-grid generation pass more important than draw-call count.

Per-cell storage	Bytes	Purpose
lifeCanvas	1	Logical CA state
targetRgb	12	Three float target channels
displayRgb	12	Three float presentation channels
texCanvasBuffer	3	RGB8 upload staging
Approximate total	28	Excludes palette, dirty structs, GPU texture, PBOs, and simulation scratch

18. Risk register

The following items are renderer-specific risks derived from the current implementation. Priority reflects likely correctness or maintenance impact in the current product, not hypothetical engine ambitions.

Priority	Risk	Why it matters	Evidence	Targeted response
P1	Silent CommandQueue overflow	Commands disappear with no signal; dirty/upload flags may already be cleared.	CommandQueue::Submit; Canvas and GLString pending flags	Return acceptance status; assert/log in Debug; preserve dirty flags on rejection.
P1	Update completion is assumed at enqueue time	A failed or rejected upload has no retry path.	Canvas::AppendCommands and GLString::AppendCommands	Acknowledge enqueue success or clear pending state only after accepted submission.
P2	Logical window size used as GL viewport	Scaled displays may render into only part of the framebuffer.	RenderWindow resize vs Renderer::RenderScene	Expose framebuffer dimensions and use them for SetViewport.
P2	Pipeline cache can be invalidated externally	Raw GL or immediate fallback could change state without updating _currentGLState.	GLDevice reset covers binds but not PipelineState	Keep production token-only; optionally reset/apply a known baseline per submission.
P2	Scissor enable has no paired disable token	Future scissor use could leak into later draws.	SetScissor always glEnable; no disable command	Model scissor enable in PipelineState or add explicit disable.

Priority	Risk	Why it matters	Evidence	Targeted response
P2	Shader failures are not propagated	Invalid programs can be enrolled and referenced later.	GLShaderProgram compile/link flow	Return structured creation status and reject unusable registry entries.
P3	Second empty submission every frame	Small avoidable CPU walk and confusing backend semantics.	RenderScene submit/clear plus EndFrame submit	Make submission ownership explicit: RenderScene or EndFrame, not both.
P3	Full-size PBO orphan for dirty updates	Small dirty rectangles still refresh full PBO capacity.	GLTexture::UpdateSubImage	Profile first; consider rect-sized staging only when large grids justify it.
P3	AABB dirty representation	Distant changes enlarge fade and upload work.	DirtyRect include semantics	Retain until profiling shows sparse lists or tiled dirty regions are worthwhile.
P3	Untyped handles and string uniforms	Wrong-type IDs and renamed uniforms fail at runtime.	unsigned long IDs; char[32] names	Use lightweight typed handles or cached uniform IDs only if renderer complexity grows.
P4	Dormant render-pass scaffolding	Headers can overstate capability and increase maintenance burden.	RenderPass, framebuffer vectors, ignored command types	Document as inactive or remove when cleanup is explicitly scoped.
P4	Redundant GLEW initialization	Startup duplication and confusing ownership.	RenderWindow and GLBackend both call glewInit	Assign GL loader initialization to one layer.

19. Active features versus scaffolding

Capability	Status	Current evidence
Ordered Scene rendering	Active	Scene vector plus Renderer linear traversal
Token-first production drawables	Active	Canvas, CommandLine, GLString, SplashText
OpenGL backend	Active	GLBackend and GLDevice
Headless backend	Active	MockBackend and CSimTests
Dirty RGB Canvas texture	Active	Canvas RGB staging and UpdateTexture
PBO texture updates	Active	GLTexture::UpdateSubImage
Dynamic UI VBO updates	Active	GLMesh::UpdateVertexData
Bind-state suppression	Active within one submit	GLDevice program, VAO, texture, viewport trackers
Immediate fallback	Supported but non-production	Drawable::Draw after token submit
Render pass objects	Scaffolding	Stored by Renderer but not used by RenderScene
Framebuffers	Scaffolding	Stored IDs without active binding path
Uniform buffer tokens	Declared, ignored	GLDevice no-op case
DrawBatched	Declared, ignored	GLDevice no-op case
DrawInstanced	Interpreter exists, unused	GLDevice maps to glDrawArraysInstanced
Scissor	Interpreter exists, unused	SetScissor enables test and sets rect
Multiple production GPU APIs	Not implemented	GL is sole real backend
Render graph or pass scheduling	Not implemented	No dependency or attachment execution model
Per-cell geometry or instancing	Not used	Canvas texture on one quad

20. Recommended improvement sequence

The renderer does not need a redesign. The following sequence improves correctness and clarity while preserving the current token architecture.

1. Make command acceptance explicit

Change CommandQueue::Submit and IBackend::PushToCommandQueue to return success or a small status. Preserve pending-update flags if enqueue fails. Add Debug assertions and a release warning/counter.

2. Define one submission owner

Choose either RenderScene or EndFrame as the normal submit point. A practical option is: BeginFrame clears, RenderScene only emits, EndFrame submits once and swaps.

3. Correct framebuffer sizing

Add framebuffer dimensions to IRenderWindow and use them for SetViewport while retaining logical window dimensions for UI and camera coordinates.

4. Close state leaks

Represent scissor enable/disable explicitly and keep raw OpenGL out of production drawables. If immediate fallback stays, document its state-restoration obligations.

5. Improve creation errors

Have shader, mesh, and texture creation report failure; avoid inserting unusable resources into registries; include the failing handle and asset path in logs.

6. Add proportional observability

Expose queue high-water count, rejected-command count, registry sizes, draw count, texture-upload bytes, and buffer-upload bytes in Debug builds.

7. Clean inactive surface area

Once behavior is stable, either remove unused render-pass/framebuffer APIs or mark them clearly experimental. Do not build pass scheduling merely to justify the types.

8. Profile before changing Canvas upload structures

The current AABB dirty region and full-size PBOs are reasonable at 80 x 60. Revisit them only with large-grid evidence.

20.1 Changes specifically not recommended now

- A general render graph.
- Opaque, transparent, UI, and debug pass objects without a concrete ordering problem.
- An ECS or scene graph for the dense Canvas.
- Per-cell meshes, instancing, or draw-indirect replacement for the current texture quad.
- Vulkan or Metal abstractions without a committed second real backend.
- Multithreaded command generation while command payloads still borrow raw pointers.

21. Detailed token sequence reference

21.1 Idle Canvas frame

```
0  SetViewport
1  SetPipelineState        # frame default
2  ClearScreen
3  SetPipelineState        # Canvas state
4  SetShader
5  SetMesh
6  SetTexture
7  SetUniformMat4          # uMVP
8  SetUniformInt           # uDisplayTexture = 0
9  DrawIndexed             # 6 indices
```

21.2 Dirty Canvas frame

```
0  SetViewport
1  SetPipelineState        # frame default
2  ClearScreen
3  UpdateTexture           # RGB dirty rectangle
4  SetPipelineState        # Canvas state
5  SetShader
6  SetMesh
7  SetTexture
8  SetUniformMat4          # uMVP
9  SetUniformInt           # uDisplayTexture = 0
10 DrawIndexed             # 6 indices
```

21.3 Warm cached GLString

```
SetPipelineState            # depth off, blend on
SetShader
SetMesh
SetUniformVec2              # u_resolution
SetUniformVec2              # u_position
SetUniformVec2              # u_scale
DrawIndexed                 # cachedNumQuads * 6
```

21.4 Dirty GLString

```
SetPipelineState
SetShader
SetMesh
UpdateBuffer                # cachedNumQuads * 4 vertices
SetUniformVec2              # u_resolution
SetUniformVec2              # u_position
SetUniformVec2              # u_scale
DrawIndexed
```

21.5 Visible CommandLine

```
SetPipelineState            # depth off, blend on
SetShader
SetMesh
SetUniformVec2              # u_resolution
SetUniformVec2              # u_scale
UpdateBuffer                # one packed panel/text batch
DrawIndexed                 # drawQuads * 6
```

22. Source map

The following files are the shortest route for future renderer investigation. Line numbers reflect the inspected 2026-08-02 working tree and may drift after edits.

Concern	Primary file	Key region
Application loop	CSim/Source/App/CellMain.cpp	31-39
Service construction	CSim/Source/Engine/Illumo.cpp	101-120
Frame orchestration	CSim/Source/Engine/Illumo.cpp	152-181
Module interface	CSim/Source/Engine/IModule.h	1-22
Scene list	CSim/Source/Rendering/Scene.h	7-34
Drawable contract	CSim/Source/Rendering/Drawable.h	6-48
Renderer construction/enrollment	CSim/Source/Rendering/Renderer.h	68-183
Renderer token helpers	CSim/Source/Rendering/Renderer.h	210-361
RenderScene	CSim/Source/Rendering/Renderer.h	367-415

Concern	Primary file	Key region
Token proof	CSim/Source/Rendering/Renderer.h	418-499
Command model	CSim/Source/Rendering/RenderCommand.h	4-146
Queue capacity	CSim/Source/Rendering/CommandQueue.h	5-29
Backend interface	CSim/Source/Rendering/IBackend.h	8-33
GL registry/backend	CSim/Source/Rendering/OpenGL/GLBackend.cpp	18-165
GL command interpreter	CSim/Source/Rendering/OpenGL/GLDevice.cpp	4-347
GL mesh wrapper	CSim/Source/Rendering/OpenGL/GLMesh.h	7-199
GL texture/PBO	CSim/Source/Rendering/OpenGL/GLTexture.h	9-278
GL shader wrapper	CSim/Source/Rendering/OpenGL/GLShaderProgram.h	10-104
Canvas data/presentation	CSim/Source/Game/Canvas.cpp	13-577
Canvas shaders	CSim/Shader/canvas_vertex.glsl; canvas_frag.glsl	entire files
GLString	CSim/Source/Rendering/GLString.cpp	8-296
CommandLine	CSim/Source/Services/CommandLine.cpp	14-640
SplashText	CSim/Source/Rendering/SplashText.h/.cpp	entire files
Mock backend	CSim/Source/Rendering/Mock/MockBackend.h	1-250
Renderer tests	CSim/Source/Tests/TestRendererE2E.cpp	173-345
UI token tests	CSim/Source/Tests/TestUITokens.cpp	17-388
Canvas tests	CSim/Source/Tests/TestCanvasDomain.cpp	entire file

23. Final assessment

CSim's renderer is coherent because it solves the actual product problem rather than modeling a hypothetical general engine. The application needs a cellular-automata surface, a few overlays, camera transforms, dynamic text, and testable behavior. One ordered token stream, one production OpenGL backend, and a recording MockBackend meet those needs with understandable ownership and low draw-call cost.

The design's next maturity step is not more abstraction. It is stronger contracts around queue capacity, update acceptance, viewport units, shader creation, and state ownership. Those changes would make the existing architecture more reliable without altering its essential shape. Canvas should remain a one-quad texture presentation until large-grid measurements show that CPU fade memory or dirty-region behavior is a real bottleneck.

RECOMMENDED ARCHITECTURAL POSTURE

Preserve: module-owned composition, per-frame Scene list, token-first drawables, opaque backend handles, one Canvas quad, MockBackend tests, and shutdown-owned GL resources. Improve: explicit enqueue results, one submit boundary, framebuffer-aware viewport, state-leak prevention, structured resource errors, and lightweight renderer telemetry.

End of report. No repository implementation files were modified to produce this analysis; only the requested PDF artifact and temporary generation files were created.